

Move

Move on Sui enables secure smart contracts and high-performance on-chain object management



Move Concepts

Move is a secure, open-source language for building smart contracts and managing on-chain objects. On Sui, Move is enhanced with an object-centric model and parallel execution to achieve high performance, fast finality, and scalable applications.

Key differences

- Sui uses its own object-centric global storage
- Addresses represent object IDs
- Sui objects have globally unique IDs
- Sui has module initializers
- Sui has unique use cases for the entry keyword

Move Module

A module is a unit of code that defines:

- Data structures (struct)
- Functions (public/ public(package) / entry)
- Business logic/Constants and imports

Modules encapsulate data and logic and control which functions are accessible from outside.

Key idea: On a blockchain like Sui, a module is the core unit of smart contract logic.

Example:

```
module hello_world::hello_world;  
// imports, structs, functions, constants go here
```

Move Package

A package is a collection of one or more Move modules along with configuration files (*Move.toml*).

- *Move.toml* manifest (configuration: name, dependencies, addresses)
- *sources/* with one or more Move modules
- optional *tests/*, *examples/*, etc.

Key idea: Package = Deployable project.

```
a_move_package
├── Move.toml    (manifest)
├── sources      (your modules)
└── tests        (optional)
```

Package Manifest

The Move.toml file in the root of a Move package. It describes the package and its dependencies and is written in TOML format.

- [package] – metadata about the package (name, version, edition, etc.).
- [dependencies] – other Move packages this package depends on (git or local paths).
- [addresses] – named address aliases you can use in code.
- [dev-dependencies] – dependencies used only in dev / test modes.
- [dev-addresses] – overrides for named addresses in dev / test modes.

Hello World



Move Basic

Object model on Sui

An object is the fundamental unit of storage on Sui. Every resource, asset, or piece of data on-chain is an object, addressable by a globally unique ID. Each object has:

- ID – a globally unique identifier that never changes.
- Owner – either a Sui address or another object (or it can be shared / immutable, depending on ownership type).
- Version number – increments every time the object is changed.
- Metadata – for example, the digest (hash) of the last transaction that used the object.

Move Basic

Object ownership

Every object has an owner field that dictates who can use it in transactions. This field can contain one of the following options:

- Address-owned: Owned by a specific address (account or object). Only the owner can access or modify it.
- Immutable: No owner. Anyone can read, but it cannot be changed, transferred, or deleted.
- Shared: Accessible by anyone. State changes are processed through consensus.
- Party: Accessible only by a defined group (Party) and managed through consensus.

Move Basic

Object and Package Versioning

Object and package versioning on Sui describes how on-chain data and code evolve over time and how those versions are referenced.

Object Versioning

- Each object is referenced as (ID, version)
- When modified, the object keeps the same ID but gets a higher version
- Only the latest version can be used in new transactions
- Versions increase monotonically, creating a clear history

Package Versioning

- Packages are immutable once published
- User packages: each upgrade creates a new package ID (new version)
- Framework packages (0x1, 0x2): keep the same ID, version increases via system upgrade

Move Basic

Custom Types with Struct

Custom types with struct are user-defined data types you create to model your own concepts (like Artist, Record, or on Sui, on-chain objects). They are defined with the struct keyword and a set of named fields.

```
/// A struct representing an artist.
public struct Artist {
    /// The name of the artist.
    name: String,
}

/// A struct representing a music record.
public struct Record {
    /// The title of the record.
    title: String,
    /// The artist of the record. Uses the `Artist` type.
    artist: Artist,
    /// The year the record was released.
    year: u16,
    /// Whether the record is a debut album.
    is_debut: bool,
    /// The edition of the record.
    edition: Option<u16>,
}
```

```
let mut artist = Artist {
    name: b"The Beatles".to_string()
};

// Access the `name` field of the `Artist` struct.
let artist_name = artist.name;
// Access a field of the `Artist` struct.
assert_eq!(artist.name, b"The Beatles".to_string());
// Mutate the `name` field of the `Artist` struct.
artist.name = b"Led Zeppelin".to_string();
// Check that the `name` field has been mutated.
assert_eq!(artist.name, b"Led Zeppelin".to_string());
```

Move Basic

Understanding Abilities in Move

Abilities are a typing feature that controls what actions are permissible for values of a given type. This is the backbone of Move's "resource safety," ensuring that on-chain objects are handled securely. There are four primary abilities:

1. key

- Definition: Allows a struct to be used as a Sui Object.
- Requirement: On Sui, any struct with the key ability must have an id: UID as its first field.
- Usage: It allows the object to be stored in the global storage and indexed by a globally unique ID.

2. store

- Definition: Allows a value to be stored inside another struct (Wrapping).
- Usage: Essential for Dynamic Fields. If you want to attach data to an object or nest one struct inside another, that data type must have the store ability.

Move Basic

Understanding Abilities in Move

3. copy

- Definition: Allows the value to be duplicated or copied.
- Security Note: Most high-value assets (like a unique NFT or a Player object) do not have the copy ability to prevent unauthorized duplication.
- Usage: Common for simple data types like coordinates or small metadata.

4. drop

- Definition: Allows the value to be ignored or "dropped" at the end of a function scope.
- Logic: If a struct does not have drop, it must be explicitly handled (consumed, stored, or returned). This prevents developers from accidentally "losing" important assets in the code.

Move Basic

Understanding Abilities in Move

// A unique Game Asset (Cannot be copied or dropped)

public struct Hero has key, store {

*id: UID, *// Required for 'key' [cite: 69]**

hp: u64,

experience: u64,

}

// Simple data that can be copied or deleted

public struct Stats has copy, drop, store {

level: u8,

rank: u64,

}

Move Basic

Dynamic fields

Let you attach flexible key-value data to an object after creation, using the object's UID. They allow storing dynamic or heterogeneous data without changing the original struct.

Types

- Dynamic Field (`dynamic_field`)
 - Value: any type with store and Objects become wrapped (not directly accessible by ID)
- Dynamic Object Field (`dynamic_object_field`)
 - Value must be an object (key, store) and Object remains accessible by its ID

Operations

- `add` — attach
- `borrow` / `borrow_mut` — read or update
- `remove` — detach

WORKSHOP TIME

